PISCINE MAKERS · EMBEDDING LAB

# Semantic Agent Engine

Une mémoire qui sait ce qui a été décidé,
et ce qui est encore vrai.

Architecture v3 · Jonathan Chayna · octobre 2026

**Architecture v3** — la réunion est immuable ; les messages d'une décision forment un nœud ; `as_of` relit l'état passé.

## L'idée en une minute

En réunion, tout n'est pas décidé tout de suite : on propose, on hésite, on change d'avis. Une recherche classique retrouve les phrases qui **se ressemblent**, mais pas forcément celle qui est **encore vraie aujourd'hui**. Ce moteur lit chaque échange, le range dans une mémoire organisée, regroupe les messages d'une même décision dans un **nœud** — le dernier message dit l'état courant — et peut relire l'état à une date passée sans jamais réécrire la réunion.

### Il comprend

Une proposition n'est pas une décision. « On pourrait peut-être… » ne déclenche rien ; « c'est validé », oui.

### Il se souvient

Chaque information garde sa date, la personne qui l'a dite et la phrase exacte d'origine.

### Il propose, il n'exécute pas

E-mails, tâches, événements : toujours des brouillons à confirmer. Aucune adresse n'est inventée.

### L'image à retenir : un atelier bien rangé

On ne retourne pas tout l'atelier pour trouver une clé à molette : chaque outil est rangé au bon endroit **dès son arrivée**. Ici, chaque information est rangée à côté de celles qui lui sont liées, au moment où elle arrive. Retrouver devient rapide, et quand une information change, un nouveau message rejoint le nœud ; l'ancien reste exactement ce qui a été dit.

## Un seul cœur, deux usages

### Réunions et messages

Le sujet de la piscine : retrouver une décision, qui s'est engagé, ce qui a changé entre deux réunions, préparer un dossier ou un brouillon d'e-mail.

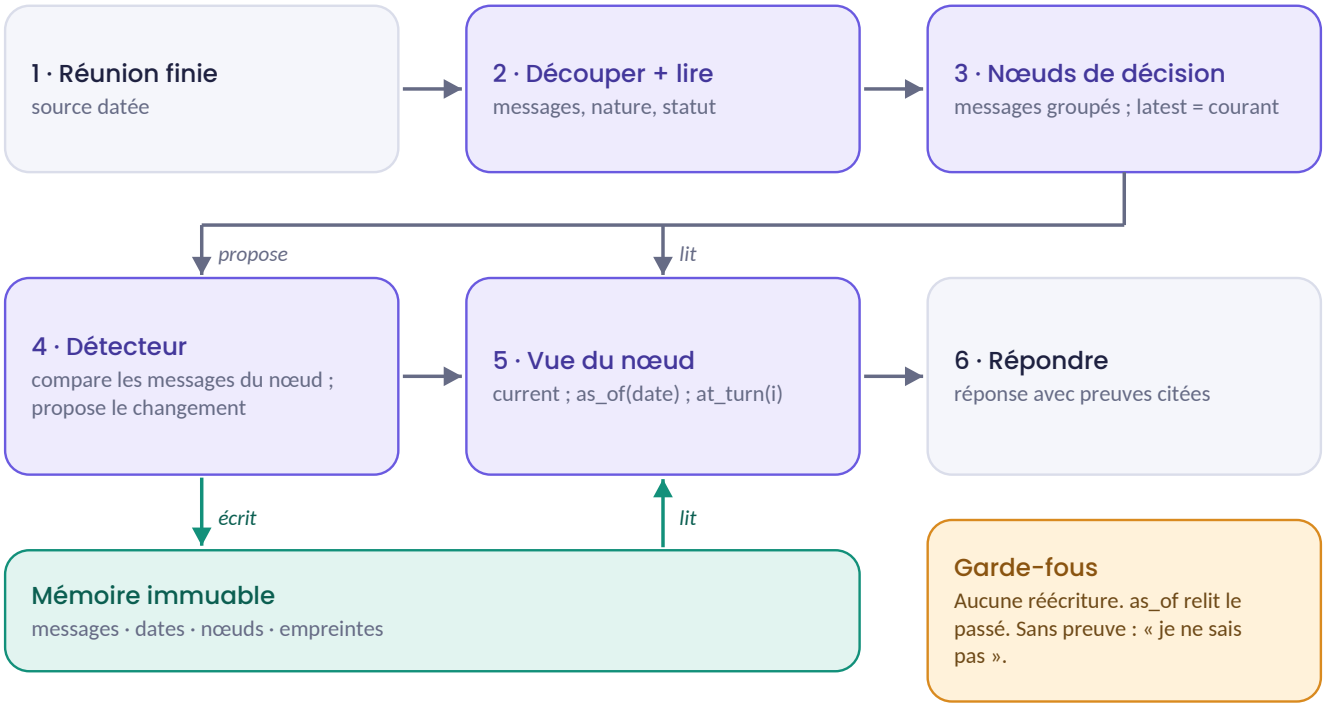
### Documents et code

Le même moteur sert de recherche générale : seul le découpage change (un paragraphe cohérent, une fonction). La mémoire, les liens et la recherche restent identiques.

# 1 Le parcours d'un message

Voici ce qui se passe quand un message arrive. Exemple utilisé dans tout le document :

> *« Pour le projet X, on repousse le lancement à décembre. Au fait, où en est le back-end ? »*




### Sur l'exemple

- **Morceau A** : « on repousse le lancement à décembre ». C'est une information (une décision). Elle est rangée en mémoire et rejoint le nœud « lancement » ; l'ancienne date reste telle qu'elle a été dite.
- **Morceau B** : « où en est le back-end ? ». C'est une question : on cherche dans la mémoire, **après** avoir enregistré A, pour que la réponse tienne compte de ce qui vient d'être dit.

# 2 Découper par sujet

Un morceau = la plus grande portion qui parle encore de la même chose

### Documents

On regroupe les phrases voisines qui parlent de la même chose. Cinq phrases sur un même sujet : un seul morceau. Une phrase sur une voiture puis une sur un ballon : deux morceaux.

### Conversations

On coupe dès que le sujet ou l'intention change, même au milieu d'une prise de parole. Dans l'exemple, une décision puis une question : deux morceaux. Un « OK, validé » très court reste collé au message précédent.

### Code

Une fonction = un morceau. Ce sont de simples règles de lecture du code, sans intelligence artificielle.

**Toujours :** même quand un morceau regroupe plusieurs phrases, on garde chaque phrase d'origine comme **preuve**. Un gros morceau ne doit pas noyer un détail comme une date ou un nom.

## 3 La mémoire

Ce qu'on garde, et comment on le range pour aller vite

### Pour chaque morceau, on garde

- le **texte** et la **phrase exacte** d'origine (la preuve)
- sa **nature** : décision, action, engagement, question, réponse, fait, risque, idée
- son **statut** : proposé, confirmé, annulé — et le **nœud** auquel il appartient
- la **date**, la **personne** qui parle, le **responsable** et l'**échéance**
- la **source** (réunion, document) et le **projet**
- des **liens** vers les autres messages du même nœud : prolonge, confirme, répond à
- une **empreinte de sens** : une suite de nombres qui représente ce que veut dire le morceau

### Comment on le range

**Un cahier** (base SQLite). Tout y est écrit : texte, dates, personnes, statuts, liens. Filtrer par projet ou par date est instantané, et la recherche par mots exacts est déjà intégrée.

**Un index de sens** (FAISS). Deux textes de sens proche ont des empreintes proches ; l'index retrouve les plus proches en quelques millisecondes. Chaque entrée renvoie au numéro du morceau dans le cahier.

**Règle d'or : on n'efface jamais.** Un nœud garde tous ses messages ; `as_of(date)` relit l'état d'une date passée. L'historique reste disponible pour comparer deux réunions.

**Pourquoi pas de simples fichiers JSON ?** Ils sont lents à filtrer et à mettre à jour quand la mémoire grandit. Le JSON reste utile *à l'intérieur* du cahier, pour les informations supplémentaires de forme libre. Chaque morceau a deux empreintes : une **stable** (le sens du morceau seul) et une **enrichie par le contexte**. **Vérifié :** l'empreinte enrichie perd sur données réelles → la stable reste la seule active.

## 4 Retrouver la bonne information

Six étapes, de la question à la réponse prouvée

| | |
|---|---|
| 1 | **Comprendre la demande**<br>Un petit modèle rapide devine ce que veut la personne : répondre, comparer, faire un dossier, écrire un e-mail, créer une tâche, planifier, ou demander une précision. Il repère aussi qui, quoi, pour quand. |
| 2 | **Chercher de deux façons**<br>Par le **sens** (retrouve les reformulations) et par les **mots exacts** (noms, dates, chiffres). Les deux listes sont fusionnées. |
| 3 | **Filtrer selon la demande**<br>Réponse : seulement l'information en vigueur. Comparaison : l'ancienne et la nouvelle, classées par date. Dossier : tout, avec les messages et les changements proposés signalés. |
| 4 | **Départager**<br>Un second modèle relit la question avec chaque candidat et les classe du plus au moins pertinent. |
| 5 | **Lire l'état du nœud**<br>Le nœud garde ses messages dans l'ordre. `current` est le dernier ; `as_of(date)` ou `at_turn(i)` relit l'état à ce moment-là, sans rien réécrire. |
| 6 | **Répondre avec preuves**<br>Par défaut, un gabarit rédige la réponse en citant chaque affirmation : **réponse par gabarit citable par défaut ; réécriture LLM optionnelle, validée par un vérificateur**, qui renvoie toute réécriture contenant un nom ou un nombre inventé. Pas assez de preuve : « je n'ai pas assez d'information ». Pour un e-mail, le destinataire n'est retenu que s'il est clair, récent et unique ; sinon on demande. |

## 5 Mises à jour

Vite tout de suite, en profondeur plus tard

### Tout de suite (instantané)

- découper et comprendre le nouveau message
- l'enregistrer dans la mémoire
- l'ajouter au nœud de sa décision
- ajouter le message au nœud ; le détecteur propose le changement, il ne réécrit rien

### En arrière-plan (par lots, chaque nuit)

- construire les nœuds de décision et la file de revue du détecteur
- reconstruire l'index de sens
- **audit de calibration** : gare aux seuils périmés ; l'audit recalibre et propose, sans jamais remplacer le modèle livré
- proposer un ré-entraînement quand assez de données ont été corrigées

C'est surtout utile pour les **messages**, qui arrivent en continu : la réaction est immédiate, le grand rangement se fait par lots, sans jamais ralentir la conversation.

## 6 Ce qu'on entraîne, et sur quoi

Chaque modèle a un rôle précis et se mesure séparément

| Rôle | Point de départ | On l'entraîne sur… | On mesure… |
|---|---|---|---|
| **Découpeur** où couper | Règle de similarité entre phrases, puis petit modèle multilingue | Paires de phrases voisines « même sujet / sujet différent », fabriquées en collant des extraits de réunions différentes, + exemples annotés | Qualité des coupures |
| **Lecteur** nature, statut, qui/quoi/quand, type de demande | CamemBERT (moderncamembert-base), licence MIT vérifiée | Conversations françaises annotées : réunions publiques + dialogues synthétiques vérifiés à la main | F1 par catégorie ; recours à un LLM si « pas sûr » |
| **Moteur de sens** empreintes | Base déterministe par hachage (aucun téléchargement, aucun GPU) ; e5-base pour les vérifications en français réel ; bge-m3 jamais mesuré | Paires question / bon passage + « faux amis » (même sujet, mauvais statut, personne ou date) | Recall@K, MRR, nDCG ; score sur les faux amis. **Version fine-tunée entraînée :** gagne ses propres tests, perd le test final → base gelée conservée |
| **Nœuds + détecteur** groupe, puis propose le changement | Regroupement par sujet (déterministe) + juge 7B en vérificateur sur des paires candidates | 53 paires étiquetées à la main (024c, 047a) ; les quatre autres réunions restent à annoter | Reach, pureté, précision/rappel des lignes et des courants |
| **Trieur final** classement | bge-reranker-v2-m3 prêt à l'emploi | Rien au départ. Entraîné seulement si les cas difficiles le justifient | Gain sur les cas difficiles |
| **Couche « concepts »** testée, désactivée | Petit réseau qui regarde tous les morceaux ensemble | Flux simulés : une info arrive, une autre la remplace | Mesurée sur données réelles en français : elle perd, donc elle reste désactivée. Preuve que les règles de jugement fonctionnent |
| **Rédacteur** réponses, dossiers | Gabarit qui cite chaque affirmation (par défaut) ; réécriture LLM optionnelle (drapeau) | Aucun entraînement | Citations exactes ; vérificateur qui rejette tout nom ou nombre inventé |

**Lecteur vérifié :** Laya écarté (modèle de dialogue, pas un encodeur) ; CamemBERT retenu, licence MIT vérifiée.

## 7 Comment cela répond au sujet

Les quatre difficultés du sujet, et la pièce qui s'en occupe

| Difficulté du sujet | Ce qui s'en occupe |
|---|---|
| **Pertinence**<br>retrouver même avec d'autres mots | Moteur de sens + recherche par mots exacts + tri final. |
| **Précision sémantique**<br>proposition, décision, annulation… | Lecteur de statut + « faux amis » à l'entraînement (même sujet, mauvais statut). |
| **Contexte**<br>plusieurs réunions, dates, versions | Nœuds de décision + `as_of`/`at_turn` + détecteur de changement (revue) + mémoire datée et sourcée. |
| **Intention et destinataire**<br>quelle action, pour qui | Lecteur de demandes + règle fixe pour le destinataire (clair, récent, unique) + droit de demander ou de dire « je ne sais pas ». |

## 8 Comment on saura que ça marche

- **Retrouve-t-on la bonne information ?** Recall@K et MRR, comparés à une recherche classique par mots et à un modèle de sens non entraîné.
- **Distingue-t-on proposition et décision ?** Score par catégorie, avec une attention aux négations, aux dates et aux nombres.
- **Le nœud retrace-t-il l'état à chaque tour ?** `at_turn` et `as_of` sur les paires d'or : 40/40, et zéro message réécrit.
- **Choisit-on le bon destinataire, ou demande-t-on ?** Précision, taux de faux destinataires, abstentions justifiées.
- **Les réponses sont-elles prouvées ?** Citations exactes et part d'affirmations sans preuve.

### Le test décisif : la version en vigueur

On insère « lancement en décembre » (1er octobre), puis « finalement en janvier » (5 octobre), puis quelques messages sans rapport, et on demande : « quand est le lancement ? ». **Résultat mesuré :** la bonne version sort en premier dans **0,6616** des cas en moyenne (System B), contre **0,2915** pour la recherche classique, sur des projets jamais vus (+0,3629, 3 tirages sur 3). Le test décembre → janvier figure dans le rapport : la géométrie seule échoue, la couche de relations corrige. **En v3 :** la réunion reste la vérité de sa date ; `at_turn`/`as_of` relisent l'état passé (40/40 sur les paires d'or) et le détecteur propose les changements dans une file de revue. **Honnêtement :** l'objectif absolu de 0,90 n'est pas atteint ; le détecteur n'est étiqueté que sur 2 réunions (P 0,82–1,00, R 0,33–0,57) ; un scan ciblé sur les quatre autres ne retrouve que 2/21 paires d'or, et l'étiquetage des sujets sur parole brute reste ouvert.

## 9 Preuves

Montré, pas promis

### Les démos passent

Les 7 démonstrations passent sur le corpus synthétique, et 10/10 sur trois vraies réunions françaises.

### Tout se régénère

Chaque chiffre se régénère à partir des artefacts, avec une seule commande.

### Licences vérifiées

Tous les modèles et tous les jeux de données ont une licence vérifiée.

| Mesure | Résultat |
|---|---|
| **Version en vigueur en premier (projets jamais vus)** | 0,2915 → 0,6616 (+0,3629 ; 3 tirages sur 3) face à la recherche classique |
| **v3 — messages rangés dans un nœud** | 99,5 % ; 25/40 paires d'or dans un nœud ; `at_turn(new)` = 40/40 ; 0 mutation |
| **v3 — courant et détecteur (2 réunions étiquetées)** | courants de nœud P 0,90 / R 0,42 ; lignes P 0,82–1,00 / R 0,33–0,57 |
| **Objectif absolu 0,90** | Non atteint |
| **7 démonstrations** | Réussies sur le corpus synthétique ; 10/10 sur trois vraies réunions françaises (sujets étiquetés à la main) |
| **Moteur de sens fine-tuné** | Gagne ses propres tests, perd le test final → base gelée conservée |
| **Couche « concepts » / empreinte enrichie** | Perd sur données réelles en français → désactivée |

C'est l'exigence « protocole reproductible » du sujet, **montrée plutôt que promise**.

## 10 Plan et priorités

### Semaine 1

- Données choisies et documentées
- Schéma d'annotation
- Recherche par mots + sens, sans entraînement : le point de départ chiffré

### Semaine 2

- Entraîner le lecteur et le détecteur
- Affiner le moteur de sens
- Au moins une ablation

### Semaine 3

- Agent complet, dossiers, brouillons
- Démonstration en 7 scènes
- Rapport : résultats, erreurs, limites

**Si le temps manque, l'ordre est :** (1) mémoire + recherche hybride + lecteur + nœuds + détecteur ; (2) moteur de sens affiné ; (3) découpeur appris ; (4) couche « concepts » comme expérience comparée. Le cœur fonctionne sans la couche (4).

## 11 Limites mesurées

Ce que les mesures montrent, pas ce qu'on craignait

- **Le détecteur n'est étiqueté que sur 2 réunions sur 6.** Rappel 0,42 (8/19 paires d'or) ; 61 des 81 nœuds retombent sur le dernier message, qui peut être du bavardage.
- **Le courant d'un nœud est le dernier message, pas forcément le dernier changement.** C'est la règle demandée ; la vérité datée se lit avec `at_turn`/`as_of`.
- **3 questions sans réponse sur 12 laissent encore filtrer une réponse** au lieu d'un « je ne sais pas ».
- **La relation « répond à » n'est jamais produite** par le relieur livré.
- **L'étiquetage des sujets sur de la parole réelle brute** est le seul problème ouvert lié à la quantité de données : il faut plus de réunions annotées.

## 12 Petit lexique

Les mots techniques, en clair

| | |
|---|---|
| **Empreinte de sens (embedding)** | Une suite de nombres qui représente le sens d'un texte. Deux textes de sens proche ont des empreintes proches. |
| **Recherche par mots (BM25)** | Retrouve les textes qui contiennent les mêmes mots que la question. Très bon pour les noms, dates et chiffres. |
| **Fusion (RRF)** | Mélange deux classements (par sens et par mots) en un seul, sans avoir à régler de poids. |
| **Trieur (reranker)** | Un modèle qui lit la question et un candidat ensemble pour mieux les classer. |
| **Nœud de décision (v3)** | Groupe les messages d'une même décision. `current` est le dernier ; `as_of`/`at_turn` relit l'état passé sans réécrire. |
| **Détecteur de changement** | Compare les messages d'un nœud et propose, dans une file de revue, lequel a changé la valeur. |
| **Faux amis (hard negatives)** | Exemples qui se ressemblent mais ne veulent pas dire la même chose : même sujet, mauvais statut, mauvaise date. |
| **Entraîner (fine-tuning)** | Reprendre un modèle déjà appris et le spécialiser sur nos propres exemples. |
| **FAISS / SQLite** | Deux outils de rangement : FAISS retrouve par proximité de sens, SQLite stocke tout le reste dans un cahier. |
| **LLM** | Modèle de langage qui rédige du texte. Ici, il est optionnel et un vérificateur contrôle ce qu'il écrit. |